

Übungseinheit: Pandas Visualisierungen

Datenanalyse mit echten Gebrauchtwagendaten

Dorian Zwanzig, Claude

28. November 2025

Inhaltsverzeichnis

1	Über diese Übung	3
1.1	Lernziele	3
1.2	Voraussetzungen	3
1.3	Über das Dataset	3
1.4	Hinweis zur Bearbeitung	4
2	Vorbereitung	5
2.1	Technische Vorbereitung	5
2.2	Mentale Vorbereitung	5
2.3	Test der Umgebung	5
3	Block A – Dataset laden und erkunden	7
3.1	Aufgabe 1: Dataset laden und inspizieren	7
3.2	Aufgabe 2: Spalten auswählen	8
4	Block B – Daten transformieren	10
4.1	Aufgabe 3: Preis in Euro umrechnen	10
4.2	Aufgabe 4: Kilometerstand berechnen	11
4.3	Aufgabe 5: Fahrzeugalter berechnen	11
5	Block C – Balkendiagramme	13
5.1	Aufgabe 6: Autos pro Marke zählen	13
5.2	Aufgabe 7: Horizontales Balkendiagramm der Top-Marken	14
6	Block D – Histogramme	16
6.1	Aufgabe 8: Preisverteilung analysieren	16
6.2	Aufgabe 9: Kilometerverteilung analysieren	17
7	Block E – Scatter Plots	19
7.1	Aufgabe 10: Kilometerstand vs. Preis	19
7.2	Aufgabe 11: Fahrzeugalter vs. Preis	20
8	Block F – Gruppierte Analysen	22
8.1	Aufgabe 12: Durchschnittspreis pro Marke	22
8.2	Aufgabe 13: Durchschnittspreis nach Baujahr	23
9	Komplexaufgaben	25
9.1	Aufgabe 14: Markenanalyse	25
9.2	Aufgabe 15: Freie Analyse	26
10	Musterlösungen	28
11	Troubleshooting	36

11.1 Häufige Fehler und Lösungen	36
11.2 Tipps zur Fehlerbehebung	36
12 Abschluss und Reflexion	37
12.1 Zusammenfassung	37
12.2 Reflexionsfragen	37
12.3 Weiterführende Ideen	37

1 Über diese Übung

1.1 Lernziele

Nach Abschluss dieser Übung können Sie:

- **CSV-Dateien laden** und DataFrame-Strukturen inspizieren
- **Neue Spalten berechnen** durch arithmetische Operationen auf bestehenden Spalten
- **Häufigkeiten zählen** mit der Methode `value_counts()`
- **DataFrames filtern** mit Bedingungen (Notebook 16b)
- **Gruppierte Analysen** mit `groupby()` und `mean()` durchführen (Notebook 17)
- **Balkendiagramme** zur Darstellung von Häufigkeiten erstellen und interpretieren
- **Histogramme** zur Analyse von Verteilungen erstellen und interpretieren
- **Scatter Plots** zur Untersuchung von Zusammenhängen erstellen und interpretieren
- **Liniendiagramme** zur Darstellung von Entwicklungen erstellen
- **Erkenntnisse aus Visualisierungen** ableiten und formulieren

Zeitbedarf: ca. 120-150 Minuten **Schwierigkeitsgrad:** Einfach bis Mittel, mit Herausforderungen

1.2 Voraussetzungen

Für diese Übung sollten Sie folgende Konzepte sicher beherrschen:

- **Variablen und Datentypen** (Notebooks 04-05)
- **Listen und Dictionaries** (Notebook 06)
- **Operatoren** (Notebook 08)
- **Fehlerbehandlung** (Notebook 11) – für Debugging

Diese Übung vertieft die Konzepte aus:

- **Notebook 16a: Pandas und Visualisierungen** (DataFrame, Spaltenzugriff, `value_counts()`, Diagramme)
- **Notebook 16b: Pandas DataFrames** (Filtern mit Bedingungen, z.B. `df[df['brand'] == 'BMW']`)
- **Notebook 17: Pandas Statistik** (`groupby()`, `mean()`)

Die Aufgaben in Block A-E kommen mit den Konzepten aus Notebook 16a aus. Ab Block F (Aufgaben 12-15) werden zusätzlich Filterung (Notebook 16b) und `groupby()/mean()` (Notebook 17) benötigt — die betroffenen Aufgaben sind entsprechend gekennzeichnet.

Falls Sie unsicher sind, wiederholen Sie bitte die entsprechenden Notebooks.

1.3 Über das Dataset

In dieser Übung arbeiten Sie mit einem **realistischen Datensatz**: Gebrauchtwagendaten aus Großbritannien mit 500 Einträgen von 9 verschiedenen Automarken.

Dataset-Details:

Eigenschaft	Wert
Datei	<code>used_cars.csv</code>
Einträge	500 Fahrzeuge
Marken	Audi, BMW, Ford, Hyundai, Mercedes, Skoda, Toyota, VW, Vauxhall
Quelle	Lehrdatensatz, angelehnt an den UK Used Car Dataset (Kaggle)

Spalten im Dataset:

Spalte	Typ	Beschreibung
brand	str	Automarke
model	str	Modellbezeichnung
year	int	Baujahr
price	int	Preis in GBP (Britische Pfund)
transmission	str	Getriebeart (Manual/Automatic/Semi-Auto)
mileage	int	Kilometerstand in Meilen
fuelType	str	Kraftstoffart (Petrol/Diesel/Hybrid/Electric)
tax	int	Jahressteuer in GBP
mpg	float	Verbrauch (Miles per Gallon)
engineSize	float	Motorgröße in Litern

1.4 Hinweis zur Bearbeitung**Dateiorganisation:**

Erstellen Sie eine Python-Datei oder ein Jupyter Notebook:

- `uebung_visualisierungen.py` oder
- `uebung_visualisierungen.ipynb`

Wichtig: Die Datei `used_cars.csv` erhalten Sie zusammen mit diesem Übungsblatt (Download im Kurs). Sie muss sich im selben Ordner befinden wie Ihre Python-Datei — dann funktioniert `pd.read_csv('used_cars.csv')` ohne Pfadanpassung. Andernfalls passen Sie den Pfad entsprechend an.

2 Vorbereitung

Aufwand: ca. 10 Minuten

2.1 Technische Vorbereitung

1. Erstellen Sie einen neuen Ordner `uebung_visualisierungen`
2. Kopieren Sie die Datei `used_cars.csv` in diesen Ordner
3. Öffnen Sie Ihre Python-Entwicklungsumgebung
4. Erstellen Sie die Übungsdatei `uebung_visualisierungen.py`

2.2 Mentale Vorbereitung

In dieser Übung werden Sie:

1. **Daten laden und erkunden:** Das Dataset kennenlernen
2. **Daten transformieren:** Neue Spalten berechnen (Währung, Einheiten)
3. **Visualisierungen erstellen:** Verschiedene Diagrammtypen anwenden
4. **Erkenntnisse ableiten:** Muster und Zusammenhänge erkennen

Wichtig: Bei jeder Visualisierung geht es nicht nur um das Erstellen, sondern auch um das **Interpretieren**. Was sagt das Diagramm aus?

2.3 Test der Umgebung

Führen Sie folgenden Code aus:

```
# Umgebungstest
print("Python-Umgebung wird getestet...")

# Test: Pandas importieren
import pandas as pd
print("[OK] Pandas importiert")

# Test: Matplotlib importieren
import matplotlib.pyplot as plt
print("[OK] Matplotlib importiert")

# Test: CSV laden
df = pd.read_csv('used_cars.csv')
print(f"[OK] Dataset geladen: {len(df)} Einträge")

# Test: Erste Zeilen anzeigen
print("\n[OK] Erste 3 Zeilen:")
print(df.head(3))

print("\n[OK] Alle Tests bestanden - Sie können beginnen!")
```

Erwartete Ausgabe:

```
Python-Umgebung wird getestet...
[OK] Pandas importiert
[OK] Matplotlib importiert
[OK] Dataset geladen: 500 Einträge
```

```
[OK] Erste 3 Zeilen:
  brand model  year  price  ... fuelType  tax  mpg  engineSize
0  Audi   A1  2017  12500  ...   Petrol   150  55.4           1.4
1  Audi   A6  2016  16500  ...   Diesel    20  64.2           2.0
2  Audi   A1  2016  11000  ...   Petrol    30  55.4           1.4
```

[3 rows x 10 columns]

[OK] Alle Tests bestanden – Sie können beginnen!

Hinweis: Pandas kürzt breite Tabellen bei der Ausgabe mit ... ab. Die genaue Formatierung (Spaltenumbruch, dtype-Angaben wie `str` oder `object`) kann je nach Pandas-Version leicht abweichen — entscheidend sind die Werte.

3 Block A – Dataset laden und erkunden

Fortschritt: [##_____] 20% der Übung

Aufwand: ca. 15 Minuten

In diesem Block laden Sie das Dataset und machen sich mit seiner Struktur vertraut.

3.1 Aufgabe 1: Dataset laden und inspizieren

Schwierigkeit: [EINFACH]

Aufgabenstellung

Laden Sie das Dataset `used_cars.csv` und verschaffen Sie sich einen ersten Überblick:

1. Importieren Sie Pandas als `pd` und Matplotlib.pyplot als `plt`
 2. Laden Sie die CSV-Datei in einen DataFrame namens `df`
 3. Zeigen Sie die ersten 5 Zeilen an mit `print(df.head())`
 4. Geben Sie aus, wie viele Zeilen und Spalten das Dataset hat
-

Hilfestellungen

Problemanalyse:

- Was muss zuerst passieren? -> Bibliotheken importieren
- Wie lädt man externe Daten? -> CSV-Datei einlesen
- Wie bekommt man einen Überblick? -> Erste Zeilen und Dimensionen anzeigen

Lösungsschritte:

1. Pandas importieren mit `import pandas as pd`
2. Matplotlib importieren mit `import matplotlib.pyplot as plt`
3. CSV-Datei laden mit `pd.read_csv('used_cars.csv')`
4. Ergebnis in Variable `df` speichern
5. Mit `print(df.head())` die ersten 5 Zeilen ausgeben
6. Mit `df.shape` die Anzahl Zeilen und Spalten ermitteln

Pseudocode:

```
IMPORTIERE pandas als pd
IMPORTIERE matplotlib.pyplot als plt
```

```
df = LADE CSV-Datei 'used_cars.csv'
```

```
AUSGABE erste 5 Zeilen von df
```

```
AUSGABE "Dataset hat X Zeilen und Y Spalten"
```

Erwartete Ausgabe

	brand	model	year	price	...	fuelType	tax	mpg	engineSize
0	Audi	A1	2017	12500	...	Petrol	150	55.4	1.4
1	Audi	A6	2016	16500	...	Diesel	20	64.2	2.0
2	Audi	A1	2016	11000	...	Petrol	30	55.4	1.4
3	Audi	A4	2017	16800	...	Diesel	145	67.3	2.0
4	Audi	A3	2016	17300	...	Petrol	145	49.6	1.4

```
[5 rows x 10 columns]
```

Das Dataset hat 500 Zeilen und 10 Spalten.

3.2 Aufgabe 2: Spalten auswählen

Schwierigkeit: [EINFACH]

Aufgabenstellung

Üben Sie das Auswählen einzelner und mehrerer Spalten:

1. Wählen Sie die Spalte `brand` aus und geben Sie die ersten 10 Werte aus
 2. Wählen Sie die Spalten `brand`, `model` und `price` gemeinsam aus
 3. Geben Sie davon die ersten 5 Zeilen aus
-

Hilfestellungen

Problemanalyse:

- Wie greift man auf eine einzelne Spalte zu? -> Eckige Klammern mit Spaltenname
- Wie greift man auf mehrere Spalten zu? -> Doppelte eckige Klammern mit Liste
- Wie begrenzt man die Ausgabe? -> `.head(n)` Methode

Lösungsschritte:

1. Einzelne Spalte auswählen: `df['brand']`
2. Mit `.head(10)` auf die ersten 10 Einträge begrenzen
3. Mehrere Spalten auswählen: `df[['brand', 'model', 'price']]`
4. Mit `.head(5)` auf die ersten 5 Zeilen begrenzen
5. Beide Ergebnisse ausgeben

Pseudocode:

```
AUSGABE "Die ersten 10 Marken:"
```

```
AUSGABE df['brand'] begrenzt auf 10 Einträge
```

```
AUSGABE "Marke, Modell und Preis (erste 5 Zeilen):"
```

```
auswahl = df mit Spalten ['brand', 'model', 'price']
```

```
AUSGABE auswahl begrenzt auf 5 Zeilen
```

Erwartete Ausgabe

Die ersten 10 Marken:

```
0    Audi
1    Audi
2    Audi
3    Audi
4    Audi
5    Audi
6    Audi
7    Audi
8    Audi
9    Audi
Name: brand, dtype: str
```


Marke, Modell und Preis (erste 5 Zeilen):

	brand	model	price
0	Audi	A1	12500
1	Audi	A6	16500
2	Audi	A1	11000
3	Audi	A4	16800
4	Audi	A3	17300

4 Block B – Daten transformieren

Fortschritt: [####] 40% der Übung

Aufwand: ca. 20 Minuten

In diesem Block lernen Sie, wie Sie neue Spalten aus bestehenden Daten berechnen können. Dies ist eine wichtige Fähigkeit für die Datenanalyse.

Neue Funktion: Spalte berechnen

Sie können eine neue Spalte erstellen, indem Sie arithmetische Operationen auf bestehende Spalten anwenden:

```
df['neue_spalte'] = df['alte_spalte'] * faktor
```

Die Berechnung wird automatisch für alle Zeilen durchgeführt.

4.1 Aufgabe 3: Preis in Euro umrechnen

Schwierigkeit: [EINFACH]

Aufgabenstellung

Die Preise im Dataset sind in Britischen Pfund (GBP) angegeben. Erstellen Sie eine neue Spalte mit den Preisen in Euro:

1. Erstellen Sie eine neue Spalte `preis_eur`
 2. Berechnen Sie: `preis_eur = price * 1.17` (aktueller Wechselkurs ca. 1 GBP = 1.17 EUR)
 3. Geben Sie die Spalten `brand`, `price` und `preis_eur` für die ersten 5 Zeilen aus
-

Hilfestellungen

Problemanalyse:

- Was ist das Ziel? -> Neue Spalte mit umgerechneten Werten
- Welche Operation wird benötigt? -> Multiplikation aller Werte mit Faktor
- Wie erstellt man eine neue Spalte? -> Zuweisung mit `df['neue_spalte'] = ...`

Lösungsschritte:

1. Wechselkurs als Faktor festlegen (1.17)
2. Neue Spalte erstellen durch Multiplikation: `df['preis_eur'] = df['price'] * 1.17`
3. Mehrere Spalten auswählen für die Ausgabe
4. Mit `.head(5)` begrenzen und ausgeben

Pseudocode:

```
wechselkurs = 1.17
```

```
df['preis_eur'] = df['price'] * wechselkurs
```

AUSGABE df mit Spalten ['brand', 'price', 'preis_eur'] begrenzt auf 5 Zeilen

Erwartete Ausgabe

```
brand price preis_eur
0 Audi 12500 14625.0
1 Audi 16500 19305.0
```

2	Audi	11000	12870.0
3	Audi	16800	19656.0
4	Audi	17300	20241.0

4.2 Aufgabe 4: Kilometerstand berechnen

Schwierigkeit: [EINFACH]

Aufgabenstellung

Die Laufleistung (mileage) ist in Meilen angegeben. Für europäische Nutzer ist der Kilometerstand praktischer:

1. Erstellen Sie eine neue Spalte km
 2. Berechnen Sie: $\text{km} = \text{mileage} * 1.609$ (1 Meile = 1.609 Kilometer)
 3. Geben Sie die Spalten brand, model, mileage und km für die ersten 5 Zeilen aus
-

Hilfestellungen

Problemanalyse:

- Was ist das Ziel? -> Meilen in Kilometer umrechnen
- Welcher Umrechnungsfaktor? -> 1 Meile = 1.609 km
- Gleiche Technik wie bei Aufgabe 3? -> Ja, neue Spalte durch Berechnung

Lösungsschritte:

1. Umrechnungsfaktor festlegen (1.609)
2. Neue Spalte erstellen: `df['km'] = df['mileage'] * 1.609`
3. Relevante Spalten für Ausgabe auswählen
4. Ergebnis ausgeben

Pseudocode:

```
umrechnung_meilen_km = 1.609
```

```
df['km'] = df['mileage'] * umrechnung_meilen_km
```

```
AUSGABE df mit Spalten ['brand', 'model', 'mileage', 'km'] begrenzt auf 5 Zeilen
```

Erwartete Ausgabe

	brand	model	mileage	km
0	Audi	A1	15735	25317.615
1	Audi	A6	36203	58250.627
2	Audi	A1	29946	48183.114
3	Audi	A4	25952	41756.768
4	Audi	A3	23254	37415.686

4.3 Aufgabe 5: Fahrzeugalter berechnen

Schwierigkeit: [MITTEL]

Aufgabenstellung

Berechnen Sie das Alter jedes Fahrzeugs:

1. Erstellen Sie eine neue Spalte `alter`
 2. Berechnen Sie: `alter = 2025 - year` (aktuelles Jahr minus Baujahr)
 3. Geben Sie die Spalten `brand`, `model`, `year` und `alter` für die ersten 10 Zeilen aus
 4. **Interpretation:** Welches Baujahr kommt in den ersten 10 Zeilen am häufigsten vor?
-

Hilfestellungen**Problemanalyse:**

- Wie berechnet man das Alter? -> Aktuelles Jahr minus Baujahr
- Welche Spalte enthält das Baujahr? -> `year`
- Welche Operation? -> Subtraktion statt Multiplikation

Lösungsschritte:

1. Aktuelles Jahr festlegen (2025)
2. Neue Spalte durch Subtraktion: `df['alter'] = 2025 - df['year']`
3. Relevante Spalten auswählen
4. Mit `.head(10)` auf 10 Zeilen begrenzen
5. Ergebnis ausgeben und interpretieren

Pseudocode:

```
aktuelles_jahr = 2025
```

```
df['alter'] = aktuelles_jahr - df['year']
```

```
AUSGABE df mit Spalten ['brand', 'model', 'year', 'alter'] begrenzt auf 10 Zeilen
```

```
INTERPRETATION: Welches Baujahr ist am häufigsten?
```

Erwartete Ausgabe

	brand	model	year	alter
0	Audi	A1	2017	8
1	Audi	A6	2016	9
2	Audi	A1	2016	9
3	Audi	A4	2017	8
4	Audi	A3	2016	9
5	Audi	A1	2016	9
6	Audi	A4	2016	9
7	Audi	A3	2019	6
8	Audi	A3	2019	6
9	Audi	A1	2019	6

5 Block C – Balkendiagramme

Fortschritt: [#####_____] 50% der Übung

Aufwand: ca. 20 Minuten

Balkendiagramme eignen sich hervorragend zur Darstellung von Häufigkeiten und zum Vergleich von Kategorien.

Neue Funktion: `value_counts()`

Die Methode `value_counts()` zählt, wie oft jeder Wert in einer Spalte vorkommt:

```
anzahl = df['spalte'].value_counts()
```

Das Ergebnis ist eine Serie, sortiert nach Häufigkeit (häufigste zuerst).

5.1 Aufgabe 6: Autos pro Marke zählen

Schwierigkeit: [EINFACH]

Aufgabenstellung

Zählen Sie, wie viele Autos von jeder Marke im Dataset vorhanden sind, und stellen Sie das Ergebnis als Balkendiagramm dar:

1. Verwenden Sie `df['brand'].value_counts()` um die Anzahl pro Marke zu zählen
 2. Speichern Sie das Ergebnis in einer Variable `marken_anzahl`
 3. Geben Sie `marken_anzahl` aus
 4. Erstellen Sie ein Balkendiagramm mit `.plot(kind='bar')`
 5. Fügen Sie einen Titel hinzu: "Anzahl Fahrzeuge pro Marke"
 6. Zeigen Sie das Diagramm mit `plt.show()` an
-

Hilfestellungen

Problemanalyse:

- Was soll gezählt werden? -> Wie oft kommt jede Marke vor?
- Welche Methode zählt Häufigkeiten? -> `value_counts()`
- Wie visualisiert man Kategorien? -> Balkendiagramm (`kind='bar'`)

Lösungsschritte:

1. Spalte `brand` auswählen
2. Methode `value_counts()` anwenden
3. Ergebnis in Variable `marken_anzahl` speichern
4. Ergebnis ausgeben mit `print()`
5. Balkendiagramm erstellen mit `.plot(kind='bar', title='...')`
6. Mit `plt.show()` anzeigen

Pseudocode:

```
marken_anzahl = df['brand'].value_counts()
```

```
AUSGABE marken_anzahl
```

```
ERSTELLE Balkendiagramm von marken_anzahl  
mit Titel "Anzahl Fahrzeuge pro Marke"
```

```
ZEIGE Diagramm an
```

Erwartete Ausgabe

```
brand
Ford      90
VW        76
Vauxhall  69
Mercedes  66
BMW        55
Audi       54
Toyota     34
Skoda      32
Hyundai    24
Name: count, dtype: int64
```

Interpretation: Welche Marke ist im Dataset am häufigsten vertreten?

5.2 Aufgabe 7: Horizontales Balkendiagramm der Top-Marken

Schwierigkeit: [MITTEL]

Aufgabenstellung

Erstellen Sie ein horizontales Balkendiagramm nur für die 5 häufigsten Marken:

1. Zählen Sie die Marken mit `value_counts()`
2. Wählen Sie nur die ersten 5 mit `.head(5)`
3. Erstellen Sie ein horizontales Balkendiagramm mit `kind='barh'`
4. Fügen Sie Titel und Achsenbeschriftungen hinzu:
 - Titel: "Top 5 Automarken im Dataset"
 - X-Achse: "Anzahl Fahrzeuge"
 - Y-Achse: "Marke"
5. Verwenden Sie die Farbe `'steelblue'`

Hilfestellungen**Problemanalyse:**

- Wie begrenzt man auf die Top 5? -> `.head(5)` nach `value_counts()`
- Warum horizontal? -> Bessere Lesbarkeit bei Kategorienamen
- Welche Parameter für Styling? -> `title`, `xlabel`, `ylabel`, `color`

Lösungsschritte:

1. Marken zählen mit `value_counts()`
2. Auf 5 Einträge begrenzen mit `.head(5)`
3. In Variable `top_5_marken` speichern
4. Horizontales Balkendiagramm mit `.plot(kind='barh', ...)`
5. Alle Parameter setzen: `title`, `xlabel`, `ylabel`, `color`
6. Mit `plt.show()` anzeigen

Pseudocode:

```
top_5_marken = df['brand'].value_counts().head(5)
```

```
ERSTELLE horizontales Balkendiagramm von top_5_marken
mit Titel "Top 5 Automarken im Dataset"
```

```
mit X-Achse "Anzahl Fahrzeuge"  
mit Y-Achse "Marke"  
mit Farbe 'steelblue'
```

ZEIGE Diagramm an

Erwartetes Verhalten

Ein horizontales Balkendiagramm zeigt die 5 häufigsten Marken mit Ford an der Spitze.

Interpretation: Warum könnte Ford die häufigste Marke sein? (Hinweis: Das Dataset stammt aus Großbritannien)

6 Block D – Histogramme

Fortschritt: [#####____] 60% der Übung

Aufwand: ca. 20 Minuten

Histogramme zeigen die Verteilung von numerischen Daten. Sie helfen zu verstehen, wie Werte verteilt sind.

6.1 Aufgabe 8: Preisverteilung analysieren

Schwierigkeit: [EINFACH]

Aufgabenstellung

Analysieren Sie, wie die Preise (in Euro) verteilt sind:

1. Erstellen Sie ein Histogramm der Spalte `preis_eur`
 2. Verwenden Sie `bins=50` für eine feinere Auflösung
 3. Fügen Sie Titel und Achsenbeschriftungen hinzu:
 - Titel: "Verteilung der Fahrzeugpreise"
 - X-Achse: "Preis in EUR"
 - Y-Achse: "Anzahl Fahrzeuge"
 4. Fügen Sie ein Gitter hinzu mit `grid=True`
-

Hilfestellungen

Problemanalyse:

- Was zeigt ein Histogramm? -> Verteilung von numerischen Werten
- Welche Spalte soll visualisiert werden? -> `preis_eur`
- Was bedeutet `bins`? -> Anzahl der Klassen/Balken im Histogramm

Lösungsschritte:

1. Spalte `preis_eur` auswählen
2. `.plot(kind='hist')` für Histogramm verwenden
3. Parameter `bins=50` für 50 Klassen setzen
4. Titel und Achsenbeschriftungen hinzufügen
5. `grid=True` für Gitternetz
6. Mit `plt.show()` anzeigen

Pseudocode:

```
ERSTELLE Histogramm von df['preis_eur']
    mit bins = 50
    mit Titel "Verteilung der Fahrzeugpreise"
    mit X-Achse "Preis in EUR"
    mit Y-Achse "Anzahl Fahrzeuge"
    mit Gitter
```

ZEIGE Diagramm an

Erwartetes Verhalten

Das Histogramm zeigt eine rechtsschiefe Verteilung: Die meisten Autos kosten zwischen 8.000 und 25.000 EUR, mit einigen teuren Ausreißern über 35.000 EUR.

Interpretation: - In welchem Preisbereich liegen die meisten Fahrzeuge? - Gibt es teure Ausreißer? Was könnten das für Fahrzeuge sein?

6.2 Aufgabe 9: Kilometerverteilung analysieren

Schwierigkeit: [MITTEL]

Aufgabenstellung

Untersuchen Sie die Verteilung der Kilometerstände:

1. Erstellen Sie ein Histogramm der Spalte `km`
 2. Experimentieren Sie mit verschiedenen `bins`-Werten: 20, 50, 100
 3. Wählen Sie den Wert, der die Verteilung am besten zeigt
 4. Fügen Sie passende Beschriftungen hinzu
 5. Verwenden Sie die Farbe `'darkgreen'`
 6. Setzen Sie eine Größe von `figsize=(10, 5)`
-

Hilfestellungen

Problemanalyse:

- Wie beeinflusst `bins` die Darstellung? -> Mehr bins = feinere Details, weniger bins = gröbere Übersicht
- Welche zusätzlichen Styling-Parameter? -> `color`, `figsize`
- Was ist ein guter `bins`-Wert? -> Ausprobieren und vergleichen!

Lösungsschritte:

1. Spalte `km` auswählen
2. Histogramm mit verschiedenen `bins`-Werten testen (20, 50, 100)
3. Besten Wert auswählen (ca. 50 ist oft gut)
4. Styling-Parameter hinzufügen: `color='darkgreen'`, `figsize=(10, 5)`
5. Titel und Achsenbeschriftungen setzen
6. Mit `plt.show()` anzeigen

Pseudocode:

```
# Experimentieren mit verschiedenen bins-Werten:
# bins = 20  -> grobe Übersicht
# bins = 50  -> gute Balance
# bins = 100 -> sehr detailliert
```

```
ERSTELLE Histogramm von df['km']
    mit bins = 50
    mit Titel "Verteilung der Kilometerstände"
    mit X-Achse "Kilometerstand (km)"
    mit Y-Achse "Anzahl Fahrzeuge"
    mit Farbe 'darkgreen'
    mit Größe (10, 5)
```

ZEIGE Diagramm an

Erwartetes Verhalten

Das Histogramm zeigt, dass die meisten Fahrzeuge zwischen 10.000 und 90.000 km haben, mit einem Schwerpunkt bei etwa 25.000 bis 70.000 km.

Interpretation: - Was ist die typische Laufleistung im Dataset? - Gibt es Fahrzeuge mit sehr hoher Laufleistung?

7 Block E – Scatter Plots

Fortschritt: [#####___] 70% der Übung

Aufwand: ca. 20 Minuten

Scatter Plots (Streudiagramme) zeigen Zusammenhänge zwischen zwei numerischen Variablen.

7.1 Aufgabe 10: Kilometerstand vs. Preis

Schwierigkeit: [EINFACH]

Aufgabenstellung

Untersuchen Sie den Zusammenhang zwischen Kilometerstand und Preis:

1. Erstellen Sie einen Scatter Plot mit:
 - X-Achse: km (Kilometerstand)
 - Y-Achse: preis_eur (Preis)
 2. Verwenden Sie `kind='scatter'`
 3. Fügen Sie Titel und Achsenbeschriftungen hinzu:
 - Titel: "Zusammenhang: Kilometerstand und Preis"
 - X-Achse: "Kilometerstand (km)"
 - Y-Achse: "Preis (EUR)"
 4. Setzen Sie `alpha=0.3` für Transparenz (bei vielen Datenpunkten)
-

Hilfestellungen

Problemanalyse:

- Was zeigt ein Scatter Plot? -> Zusammenhang zwischen zwei numerischen Variablen
- Warum alpha verwenden? -> Bei vielen Datenpunkten werden Überlappungen sichtbar
- Wie definiert man die Achsen? -> `x='spalte1', y='spalte2'`

Lösungsschritte:

1. Scatter Plot auf dem DataFrame aufrufen: `df.plot(kind='scatter', ...)`
2. X-Achse festlegen: `x='km'`
3. Y-Achse festlegen: `y='preis_eur'`
4. Transparenz setzen: `alpha=0.3`
5. Titel und Achsenbeschriftungen hinzufügen
6. Mit `plt.show()` anzeigen

Pseudocode:

```
ERSTELLE Scatter Plot von df
    mit X-Achse = 'km'
    mit Y-Achse = 'preis_eur'
    mit Transparenz = 0.3
    mit Titel "Zusammenhang: Kilometerstand und Preis"
    mit X-Beschriftung "Kilometerstand (km)"
    mit Y-Beschriftung "Preis (EUR)"
```

ZEIGE Diagramm an

Erwartetes Verhalten

Der Scatter Plot zeigt eine negative Korrelation: Je höher der Kilometerstand, desto niedriger tendenziell der Preis.

Interpretation: - Gibt es einen erkennbaren Zusammenhang? - Wie würden Sie diesen Zusammenhang in einem Satz beschreiben?

7.2 Aufgabe 11: Fahrzeugalter vs. Preis

Schwierigkeit: [MITTEL]

Aufgabenstellung

Untersuchen Sie, wie sich das Fahrzeugalter auf den Preis auswirkt:

1. Erstellen Sie einen Scatter Plot mit:
 - X-Achse: `alter`
 - Y-Achse: `preis_eur`
 2. Verwenden Sie passende Beschriftungen:
 - Titel: "Wertverlust: Fahrzeugalter vs. Preis"
 - X-Achse: "Fahrzeugalter (Jahre)"
 - Y-Achse: "Preis (EUR)"
 3. Setzen Sie `alpha=0.2`, `color='red'` und `figsize=(10, 6)`
-

Hilfestellungen**Problemanalyse:**

- Was wollen wir untersuchen? -> Wertverlust über die Zeit
- Welche Spalten sind relevant? -> `alter` (X) und `preis_eur` (Y)
- Wie erwarten wir den Zusammenhang? -> Ältere Fahrzeuge sollten günstiger sein

Lösungsschritte:

1. Scatter Plot mit `df.plot(kind='scatter', ...)` erstellen
2. X-Achse: `x='alter'`
3. Y-Achse: `y='preis_eur'`
4. Styling: `alpha=0.2`, `color='red'`, `figsize=(10, 6)`
5. Beschriftungen hinzufügen
6. Mit `plt.show()` anzeigen

Pseudocode:

```
ERSTELLE Scatter Plot von df
    mit X-Achse = 'alter'
    mit Y-Achse = 'preis_eur'
    mit Transparenz = 0.2
    mit Farbe = 'red'
    mit Größe = (10, 6)
    mit Titel "Wertverlust: Fahrzeugalter vs. Preis"
    mit X-Beschriftung "Fahrzeugalter (Jahre)"
    mit Y-Beschriftung "Preis (EUR)"
```

ZEIGE Diagramm an

Erwartetes Verhalten

Der Scatter Plot zeigt deutlich den Wertverlust: Ältere Fahrzeuge sind in der Regel günstiger.

Interpretation: - Ab welchem Alter sind die meisten Fahrzeuge unter 20.000 EUR? - Wie groß ist die Preisspanne innerhalb eines Alters? Was könnte die Unterschiede erklären?

8 Block F – Gruppierte Analysen

Fortschritt: [#####_] 80% der Übung

Aufwand: ca. 25 Minuten

Mit `groupby()` können Sie Daten nach Kategorien gruppieren und dann Berechnungen durchführen.

Hinweis zur Einordnung: `groupby()` und `mean()` werden in **Notebook 17 (Pandas Statistik)** eingeführt. Die Aufgaben in diesem Block gehen also über Notebook 16a hinaus.

Neue Funktion: `groupby()` (Notebook 17)

Mit `groupby()` gruppieren Sie Daten nach einer Kategorie und berechnen dann Statistiken:

```
durchschnitt = df.groupby('kategorie')['wert'].mean()
```

Dieses Beispiel berechnet den Durchschnitt der Spalte `wert` für jede Kategorie.

8.1 Aufgabe 12: Durchschnittspreis pro Marke

Schwierigkeit: [MITTEL] **Konzepte:** `groupby()`/`mean()` — Notebook 17

Aufgabenstellung

Berechnen Sie den durchschnittlichen Preis (in EUR) für jede Automarke und visualisieren Sie das Ergebnis:

1. Gruppieren Sie nach `brand` und berechnen Sie den Mittelwert von `preis_eur`
2. Speichern Sie das Ergebnis in `preis_pro_marke`
3. Sortieren Sie die Werte absteigend mit `.sort_values(ascending=False)`
4. Geben Sie das Ergebnis gerundet auf 2 Nachkommastellen aus (`.round(2)`)
5. Erstellen Sie ein horizontales Balkendiagramm
6. Fügen Sie passende Beschriftungen hinzu

Hilfestellungen

Problemanalyse:

- Was wollen wir wissen? -> Durchschnittspreis pro Marke
- Welche Methode gruppiert Daten? -> `groupby()`
- Welche Berechnung? -> Mittelwert mit `mean()`

Lösungsschritte:

1. Nach Marke gruppieren: `df.groupby('brand')`
2. Spalte `preis_eur` auswählen: `['preis_eur']`
3. Mittelwert berechnen: `.mean()`
4. Ergebnis sortieren: `.sort_values(ascending=False)`
5. In Variable speichern und ausgeben
6. Horizontales Balkendiagramm erstellen

Pseudocode:

```
preis_pro_marke = df GRUPPIERT NACH 'brand'
                    WÄHLE 'preis_eur'
                    BERECHNE Mittelwert
                    SORTIERE absteigend
```

```
AUSGABE preis_pro_marke
```

ERSTELLE horizontales Balkendiagramm von `preis_pro_marke`
 mit Titel "Durchschnittspreis pro Marke"
 mit X-Beschriftung "Durchschnittspreis (EUR)"
 mit Y-Beschriftung "Marke"

ZEIGE Diagramm an

Erwartete Ausgabe

Durchschnittspreis pro Marke (in EUR):

```
brand
Mercedes    24187.62
BMW          23414.47
Audi         21740.77
VW           17912.70
Toyota       16969.13
Ford          15882.88
Hyundai      13975.16
Skoda         12661.59
Vauxhall     11902.46
Name: preis_eur, dtype: float64
```

Interpretation: - Welche Marken sind im Durchschnitt am teuersten? - Überrascht Sie das Ergebnis? Warum (nicht)?

8.2 Aufgabe 13: Durchschnittspreis nach Baujahr

Schwierigkeit: [MITTEL] **Konzepte:** `groupby()/mean()` — Notebook 17

Aufgabenstellung

Analysieren Sie, wie sich der Durchschnittspreis nach Baujahr entwickelt:

1. Gruppieren Sie nach `year` und berechnen Sie den Mittelwert von `preis_eur`
 2. Erstellen Sie ein **Liniendiagramm** mit `kind='line'`
 3. Fügen Sie passende Beschriftungen hinzu:
 - Titel: "Durchschnittspreis nach Baujahr"
 - X-Achse: "Baujahr"
 - Y-Achse: "Durchschnittspreis (EUR)"
 4. Fügen Sie ein Gitter hinzu
 5. Setzen Sie `figsize=(12, 5)` und `linewidth=2`
-

Hilfestellungen

Problemanalyse:

- Was wollen wir darstellen? -> Entwicklung des Preises über die Jahre
- Warum Liniendiagramm? -> Zeigt zeitliche Entwicklung/Trend
- Welche Gruppierung? -> Nach Baujahr (`year`)

Lösungsschritte:

1. Nach Jahr gruppieren: `df.groupby('year')`
2. Durchschnittspreis berechnen: `['preis_eur'].mean()`
3. In Variable `preis_pro_jahr` speichern
4. Liniendiagramm erstellen: `.plot(kind='line', ...)`

5. Styling hinzufügen: figsize, linewidth, grid
6. Beschriftungen setzen

Pseudocode:

```
preis_pro_jahr = df GRUPPIERT NACH 'year'
                    WÄHLE 'preis_eur'
                    BERECHNE Mittelwert

ERSTELLE Liniendiagramm von preis_pro_jahr
    mit Titel "Durchschnittspreis nach Baujahr"
    mit X-Beschriftung "Baujahr"
    mit Y-Beschriftung "Durchschnittspreis (EUR)"
    mit Gitter
    mit Größe (12, 5)
    mit Linienbreite 2

ZEIGE Diagramm an
```

Erwartetes Verhalten

Das Liniendiagramm zeigt einen klaren Trend: Neuere Fahrzeuge (höheres Baujahr) haben höhere Durchschnittspreise.

Interpretation: - Wie entwickelt sich der Preis von 2013 bis 2020? - In welchen Jahren steigt der Preis besonders stark?

9 Komplexaufgaben

Fortschritt: [#####_] 90% der Übung

Aufwand: ca. 25-30 Minuten

Diese Aufgaben kombinieren mehrere der gelernten Konzepte.

9.1 Aufgabe 14: Markenanalyse

Schwierigkeit: [SCHWER] **Konzepte:** DataFrame-Filterung — Notebook 16b

Aufgabenstellung

Führen Sie eine detaillierte Analyse für eine Marke Ihrer Wahl durch:

1. Wählen Sie eine Marke (z.B. BMW, Audi, VW)
2. Filtern Sie das Dataset: `marke_df = df[df['brand'] == 'IhreMarke']`
3. Geben Sie aus, wie viele Fahrzeuge dieser Marke im Dataset sind
4. Erstellen Sie mindestens 2 Visualisierungen für diese Marke:
 - Histogramm der Preisverteilung
 - Scatter Plot: Kilometerstand vs. Preis
5. Formulieren Sie 2-3 Erkenntnisse über diese Marke

Hilfestellungen

Problemanalyse:

- Wie filtert man einen DataFrame? -> Bedingung in eckigen Klammern
- Wie zählt man Einträge? -> `len(dataframe)` oder `dataframe.shape[0]`
- Welche Diagramme zeigen was?
 - Histogramm -> Verteilung einer Variable
 - Scatter Plot -> Zusammenhang zweier Variablen

Lösungsschritte:

1. Marke auswählen (z.B. 'BMW')
2. Filter anwenden: `bmw_df = df[df['brand'] == 'BMW']`
3. Anzahl ausgeben: `print(f"Anzahl: {len(bmw_df)}")`
4. Histogramm der Preise erstellen:
 - `bmw_df['preis_eur'].plot(kind='hist', ...)`
5. Scatter Plot erstellen:
 - `bmw_df.plot(kind='scatter', x='km', y='preis_eur', ...)`
6. Diagramme interpretieren und Erkenntnisse formulieren

Pseudocode:

```
# Marke wählen und filtern
gewaehlte_marke = 'BMW'
marke_df = df[df['brand'] == gewaehlte_marke]

AUSGABE "Anzahl Fahrzeuge:", Länge von marke_df

# Visualisierung 1: Preisverteilung
ERSTELLE Histogramm von marke_df['preis_eur']
    mit Titel "[Marke]: Preisverteilung"
    mit Beschriftungen
ZEIGE Diagramm an
```

```
# Visualisierung 2: Zusammenhang km vs. Preis
ERSTELLE Scatter Plot von marke_df
    mit X-Achse = 'km'
    mit Y-Achse = 'preis_eur'
    mit Titel "[Marke]: Kilometerstand vs. Preis"
ZEIGE Diagramm an

# Erkenntnisse formulieren:
# 1. In welchem Preisbereich liegen die meisten Fahrzeuge?
# 2. Gibt es einen Zusammenhang zwischen km und Preis?
# 3. Was fällt bei dieser Marke besonders auf?
```

Erwartetes Verhalten

Für BMW (Beispiel): - 55 Fahrzeuge - Preisverteilung zeigt Schwerpunkt bei 15.000-30.000 EUR - Scatter Plot zeigt typischen Wertverlust

Beispiel-Erkenntnisse: - "Die meisten BMW im Dataset kosten zwischen 15.000 und 30.000 EUR." - "Es gibt eine klare negative Korrelation zwischen Kilometerstand und Preis."

9.2 Aufgabe 15: Freie Analyse

Schwierigkeit: [SCHWER] **Konzepte:** je nach Fragestellung `groupby()/mean()` (Notebook 17) und Filterung (Notebook 16b)

Aufgabenstellung

Formulieren Sie eine eigene Fragestellung und beantworten Sie sie mit einer passenden Visualisierung:

Mögliche Fragestellungen: - Wie unterscheiden sich die Kraftstoffarten (`fuelType`) in Bezug auf den Preis? - Welche Getriebeart (`transmission`) ist bei welcher Marke häufiger? - Wie hängt die Motorgröße (`engineSize`) mit dem Preis zusammen? - Welche Marke hat die niedrigste durchschnittliche Laufleistung?

Ihre Aufgabe: 1. Wählen Sie eine Fragestellung oder formulieren Sie eine eigene 2. Überlegen Sie, welche Visualisierung am besten geeignet ist 3. Erstellen Sie die Visualisierung mit passenden Beschriftungen 4. Formulieren Sie Ihre Erkenntnisse in 2-3 Sätzen

Hilfestellungen

Problemanalyse:

- Welche Art von Frage stelle ich?
 - Vergleich von Kategorien? -> Balkendiagramm
 - Verteilung von Werten? -> Histogramm
 - Zusammenhang zweier Variablen? -> Scatter Plot
 - Entwicklung über Zeit? -> Liniendiagramm
- Welche Daten brauche ich? -> Welche Spalten sind relevant?
- Muss ich gruppieren oder filtern? -> `groupby()` oder `df[bedingung]`

Lösungsstrategien nach Fragetyp:

Option A: Kategorienvergleich (z.B. Kraftstoffarten)

1. Gruppieren nach Kategorie: `df.groupby('fuelType')`
2. Aggregieren (z.B. Durchschnitt): `['preis_eur'].mean()`
3. Sortieren: `.sort_values(ascending=False)`
4. Balkendiagramm erstellen

Option B: Zusammenhang (z.B. Motorgröße vs. Preis)

1. Scatter Plot mit `x='engineSize', y='preis_eur'`
2. Transparenz setzen: `alpha=0.3`
3. Beschriftungen hinzufügen

Option C: Kategorienverteilung (z.B. Getriebe pro Marke)

1. Häufigkeiten zählen: `df['transmission'].value_counts()`
2. Balkendiagramm erstellen

Pseudocode (Beispiel: Kraftstoffarten):

Fragestellung: Welche Kraftstoffart ist am teuersten?

Daten vorbereiten

```
preis_kraftstoff = df GRUPPIERT NACH 'fuelType'
                    WÄHLE 'preis_eur'
                    BERECHNE Mittelwert
                    SORTIERE absteigend
```

Visualisierung

```
ERSTELLE Balkendiagramm von preis_kraftstoff
    mit Titel "Durchschnittspreis nach Kraftstoffart"
    mit Beschriftungen
```

ZEIGE Diagramm an

Erkenntnisse:

- # - Welche Kraftstoffart ist am teuersten/günstigsten?
- # - Wie groß sind die Unterschiede?
- # - Überrascht das Ergebnis? Warum?

Erwartetes Verhalten

Eine sinnvolle Visualisierung, die Ihre Fragestellung beantwortet, mit: - Passender Diagrammart (Balken, Histogramm, Scatter, Linie) - Aussagekräftigen Beschriftungen - Klarer Interpretation der Ergebnisse

10 Musterlösungen

Lösung zu Aufgabe 1: Dataset laden und inspizieren

Lösung anzeigen

```
# Aufgabe 1: Dataset laden und inspizieren
```

```
# Bibliotheken importieren
import pandas as pd
import matplotlib.pyplot as plt
```

```
# CSV-Datei laden
df = pd.read_csv('used_cars.csv')
```

```
# Erste 5 Zeilen anzeigen
print(df.head())
```

```
# Anzahl Zeilen und Spalten ausgeben
zeilen, spalten = df.shape
print(f"\nDas Dataset hat {zeilen} Zeilen und {spalten} Spalten.")
```

Erklärung: - `pd.read_csv()` lädt die CSV-Datei in einen DataFrame - `df.head()` zeigt standardmäßig die ersten 5 Zeilen - `df.shape` gibt ein Tupel (Zeilen, Spalten) zurück

Verwendete Konzepte: - Import von Bibliotheken - CSV-Datei laden mit Pandas - DataFrame-Attribute (shape)

Lösung zu Aufgabe 2: Spalten auswählen

Lösung anzeigen

```
# Aufgabe 2: Spalten auswählen
```

```
# Einzelne Spalte auswählen: brand
print("Die ersten 10 Marken:")
print(df['brand'].head(10))

# Mehrere Spalten auswählen
print("\nMarke, Modell und Preis (erste 5 Zeilen):")
auswahl = df[['brand', 'model', 'price']]
print(auswahl.head(5))
```

Erklärung: - `df['spalte']` wählt eine einzelne Spalte aus (ergibt eine Series) - `df[['spalte1', 'spalte2']]` wählt mehrere Spalten aus (ergibt einen DataFrame) - `.head(n)` beschränkt die Ausgabe auf die ersten n Zeilen

Verwendete Konzepte: - Spaltenzugriff mit eckigen Klammern - Unterschied zwischen Series und DataFrame - `head()`-Methode

Lösung zu Aufgabe 3: Preis in Euro umrechnen

Lösung anzeigen

```
# Aufgabe 3: Preis in Euro umrechnen
```

```
# Neue Spalte berechnen: GBP zu EUR
df['preis_eur'] = df['price'] * 1.17
```

```
# Ergebnis anzeigen
print(df[['brand', 'price', 'preis_eur']].head())
```

Erklärung: - Mit `df['neue_spalte'] = berechnung` wird eine neue Spalte erstellt - Die Multiplikation wird automatisch auf alle Zeilen angewendet - Der Wechselkurs 1.17 wandelt GBP in EUR um

Verwendete Konzepte: - Neue Spalte durch Berechnung erstellen - Arithmetische Operationen auf Spalten - Mehrere Spalten auswählen

Lösung zu Aufgabe 4: Kilometerstand berechnen

Lösung anzeigen

```
# Aufgabe 4: Kilometerstand berechnen
```

```
# Neue Spalte: Meilen zu Kilometer
df['km'] = df['mileage'] * 1.609
```

```
# Ergebnis anzeigen
print(df[['brand', 'model', 'mileage', 'km']].head())
```

Erklärung: - 1 Meile entspricht 1.609 Kilometern - Die Berechnung erstellt eine neue Spalte `km` - Alle 500 Werte werden gleichzeitig umgerechnet

Verwendete Konzepte: - Spaltenberechnung - Einheitenumrechnung - Effiziente Vektoroperationen in Pandas

Lösung zu Aufgabe 5: Fahrzeugalter berechnen

Lösung anzeigen

```
# Aufgabe 5: Fahrzeugalter berechnen
```

```
# Neue Spalte: Alter berechnen
df['alter'] = 2025 - df['year']
```

```
# Ergebnis anzeigen
print(df[['brand', 'model', 'year', 'alter']].head(10))
```

```
# Interpretation: Baujahr 2016 kommt in den ersten 10 Zeilen am häufigsten vor
```

Erklärung: - Das Alter wird als Differenz zwischen aktuellem Jahr und Baujahr berechnet - Ein Fahrzeug von 2017 ist im Jahr 2025 also 8 Jahre alt - Die Subtraktion funktioniert auf der gesamten Spalte

Verwendete Konzepte: - Spaltenberechnung mit Subtraktion - Interpretation von Ergebnissen

Lösung zu Aufgabe 6: Autos pro Marke zählen

Lösung anzeigen

```
# Aufgabe 6: Autos pro Marke zählen
```

```
# Häufigkeiten zählen
marken_anzahl = df['brand'].value_counts()
print(marken_anzahl)
```

```
# Balkendiagramm erstellen
marken_anzahl.plot(kind='bar', title='Anzahl Fahrzeuge pro Marke')
```

```
plt.xlabel('Marke')
plt.ylabel('Anzahl')
plt.tight_layout()
plt.show()
```

Interpretation: Ford ist mit 90 Fahrzeugen am häufigsten vertreten

Erklärung: - `value_counts()` zählt die Häufigkeit jedes eindeutigen Wertes - Das Ergebnis ist bereits absteigend sortiert - `.plot(kind='bar')` erstellt ein Balkendiagramm

Verwendete Konzepte: - `value_counts()`-Methode - Balkendiagramm mit `.plot()` - Achsenbeschriftungen

Lösung zu Aufgabe 7: Horizontales Balkendiagramm

Lösung anzeigen

Aufgabe 7: Horizontales Balkendiagramm der Top-Marken

Top 5 Marken

```
top_5_marken = df['brand'].value_counts().head(5)
```

Horizontales Balkendiagramm

```
top_5_marken.plot(
    kind='barh',
    title='Top 5 Automarken im Dataset',
    xlabel='Anzahl Fahrzeuge',
    ylabel='Marke',
    color='steelblue'
)
plt.tight_layout()
plt.show()
```

Interpretation: Ford ist häufigste Marke - Ford ist eine britische Marke

und das Dataset stammt aus Großbritannien

Erklärung: - `.head(5)` begrenzt auf die ersten 5 Einträge - `kind='barh'` erstellt horizontale Balken - Die Farbe wird mit `color='steelblue'` gesetzt

Verwendete Konzepte: - Kombination von `value_counts()` und `head()` - Horizontales Balkendiagramm - Styling mit Farben

Lösung zu Aufgabe 8: Preisverteilung analysieren

Lösung anzeigen

Aufgabe 8: Preisverteilung analysieren

Histogramm erstellen

```
df['preis_eur'].plot(
    kind='hist',
    bins=50,
    title='Verteilung der Fahrzeugpreise',
    xlabel='Preis in EUR',
    ylabel='Anzahl Fahrzeuge',
    grid=True
)
plt.tight_layout()
plt.show()
```

```
# Interpretation:
# - Die meisten Fahrzeuge kosten zwischen 8.000 und 25.000 EUR
# - Die Verteilung ist rechtsschief (lange rechte Flanke)
# - Teure Ausreißer (über 35.000 EUR) sind selten, vermutlich junge
# Premium-Modelle mit wenig Kilometern
```

Erklärung: - kind='hist' erstellt ein Histogramm - bins=50 teilt die Daten in 50 Klassen ein - grid=True fügt ein Gitternetz hinzu

Verwendete Konzepte: - Histogramm zur Verteilungsanalyse - bins-Parameter für Klassenanzahl - Interpretation von Verteilungen

Lösung zu Aufgabe 9: Kilometerverteilung analysieren

Lösung anzeigen

```
# Aufgabe 9: Kilometerverteilung analysieren

# Histogramm mit optimiertem bins-Wert
df['km'].plot(
    kind='hist',
    bins=50, # 50 bietet gute Balance zwischen Detail und Übersichtlichkeit
    title='Verteilung der Kilometerstände',
    xlabel='Kilometerstand (km)',
    ylabel='Anzahl Fahrzeuge',
    color='darkgreen',
    figsize=(10, 5)
)
plt.tight_layout()
plt.show()

# Interpretation:
# - Typische Laufleistung: 10.000 - 90.000 km
# - Einige wenige Fahrzeuge haben über 100.000 km
# - Der Schwerpunkt liegt bei etwa 25.000-70.000 km
```

Erklärung: - figsize=(10, 5) setzt die Größe in Zoll (Breite, Höhe) - Verschiedene bins-Werte zeigen unterschiedliche Detailgrade - color='darkgreen' setzt die Balkenfarbe

Verwendete Konzepte: - Histogramm mit angepasster Größe - Farbgestaltung - Experimentieren mit Parametern

Lösung zu Aufgabe 10: Kilometerstand vs. Preis

Lösung anzeigen

```
# Aufgabe 10: Scatter Plot - Kilometerstand vs. Preis

# Scatter Plot erstellen
df.plot(
    kind='scatter',
    x='km',
    y='preis_eur',
    title='Zusammenhang: Kilometerstand und Preis',
    xlabel='Kilometerstand (km)',
    ylabel='Preis (EUR)',
    alpha=0.3 # Transparenz für bessere Sichtbarkeit bei vielen Punkten
```

```
)
plt.tight_layout()
plt.show()
```

```
# Interpretation:
# Es gibt eine negative Korrelation: Je höher der Kilometerstand,
# desto niedriger ist tendenziell der Preis.
# Das entspricht dem typischen Wertverlust durch Nutzung.
```

Erklärung: - kind='scatter' erstellt ein Streudiagramm - x und y definieren die Achsen - alpha=0.3 macht Punkte transparent, sodass Überlappungen sichtbar werden

Verwendete Konzepte: - Scatter Plot für Zusammenhangsanalyse - Transparenz bei vielen Datenpunkten - Interpretation von Korrelationen

Lösung zu Aufgabe 11: Fahrzeugalter vs. Preis

Lösung anzeigen

```
# Aufgabe 11: Scatter Plot - Fahrzeugalter vs. Preis
```

```
# Scatter Plot erstellen
df.plot(
    kind='scatter',
    x='alter',
    y='preis_eur',
    title='Wertverlust: Fahrzeugalter vs. Preis',
    xlabel='Fahrzeugalter (Jahre)',
    ylabel='Preis (EUR)',
    alpha=0.2,
    color='red',
    figsize=(10, 6)
)
plt.tight_layout()
plt.show()
```

```
# Interpretation:
# - Klarer Wertverlust: Ältere Fahrzeuge sind günstiger
# - Ab ca. 8 Jahren sind die meisten Fahrzeuge unter 20.000 EUR
# - Innerhalb eines Alters gibt es große Preisspannen - Marke und
# Kilometerstand erklären die Unterschiede
```

Erklärung: - Der Wertverlust von Fahrzeugen ist deutlich sichtbar - Die jüngsten Fahrzeuge im Dataset (Alter 5-6 Jahre, Baujahr 2019/2020) haben die höchsten Preise - Die Streuung innerhalb eines Alters entsteht durch Marke und Laufleistung

Verwendete Konzepte: - Scatter Plot mit Styling - Kombination mehrerer Parameter - Erkennen von Mustern und Ausreißern

Lösung zu Aufgabe 12: Durchschnittspreis pro Marke

Lösung anzeigen

```
# Aufgabe 12: Durchschnittspreis pro Marke
```

```
# Gruppierung und Mittelwertberechnung
preis_pro_marke = df.groupby('brand')['preis_eur'].mean()
```



```

# Sortieren (absteigend)
preis_pro_marke = preis_pro_marke.sort_values(ascending=False)

# Ausgabe
print("Durchschnittspreis pro Marke (in EUR):")
print(preis_pro_marke.round(2))

# Horizontales Balkendiagramm
preis_pro_marke.plot(
    kind='barh',
    title='Durchschnittspreis pro Marke',
    xlabel='Durchschnittspreis (EUR)',
    ylabel='Marke',
    color='purple'
)
plt.tight_layout()
plt.show()

# Interpretation:
# - Premium-Marken (Mercedes, BMW, Audi) haben höhere Durchschnittspreise
# - Vauxhall, Skoda und Hyundai sind im Durchschnitt am günstigsten
# - Das entspricht dem typischen Markenimage

```

Erklärung: - `groupby('brand')` gruppiert die Daten nach Marke - `['preis_eur'].mean()` berechnet den Durchschnitt pro Gruppe - `sort_values(ascending=False)` sortiert absteigend

Verwendete Konzepte: - `groupby()` für Gruppierung (Notebook 17) - Aggregatfunktion `mean()` (Notebook 17) - Sortieren von Ergebnissen

Lösung zu Aufgabe 13: Durchschnittspreis nach Baujahr

Lösung anzeigen

```

# Aufgabe 13: Durchschnittspreis nach Baujahr

# Gruppierung nach Baujahr
preis_pro_jahr = df.groupby('year')['preis_eur'].mean()

# Liniendiagramm erstellen
preis_pro_jahr.plot(
    kind='line',
    title='Durchschnittspreis nach Baujahr',
    xlabel='Baujahr',
    ylabel='Durchschnittspreis (EUR)',
    grid=True,
    figsize=(12, 5),
    linewidth=2
)
plt.tight_layout()
plt.show()

# Interpretation:
# - Klarer Aufwärtstrend: Neuere Fahrzeuge sind teurer
# - Besonders starker Anstieg ab 2017
# - Fahrzeuge von 2020 haben den höchsten Durchschnittspreis (ca. 26.000 EUR)

```

Erklärung: - `kind='line'` erstellt ein Liniendiagramm - Die Linie zeigt den zeitlichen Verlauf des Durchschnittspreises - `linewidth=2` macht die Linie dicker und besser sichtbar

Verwendete Konzepte: - Liniendiagramm für zeitliche Entwicklung - groupby() mit numerischer Gruppierung (Notebook 17) - Interpretation von Trends

Lösung zu Aufgabe 14: Markenanalyse

Lösung anzeigen

Aufgabe 14: Markenanalyse für BMW

Dataset filtern

```
bmw_df = df[df['brand'] == 'BMW']
```

Anzahl ausgeben

```
print(f"Anzahl BMW-Fahrzeuge im Dataset: {len(bmw_df)}")
```

Visualisierung 1: Preisverteilung

```
bmw_df['preis_eur'].plot(
    kind='hist',
    bins=40,
    title='BMW: Preisverteilung',
    xlabel='Preis (EUR)',
    ylabel='Anzahl',
    color='blue',
    figsize=(10, 4)
)
plt.tight_layout()
plt.show()
```

Visualisierung 2: Scatter Plot

```
bmw_df.plot(
    kind='scatter',
    x='km',
    y='preis_eur',
    title='BMW: Kilometerstand vs. Preis',
    xlabel='Kilometerstand (km)',
    ylabel='Preis (EUR)',
    alpha=0.4,
    color='blue'
)
plt.tight_layout()
plt.show()
```

Erkenntnisse:

1. Die meisten BMW im Dataset kosten zwischen 15.000 und 30.000 EUR

2. Es gibt eine klare negative Korrelation zwischen km und Preis

3. Einige BMW mit geringem km-Stand kosten über 30.000 EUR (neuere Modelle)

Erklärung: - `df[df['brand'] == 'BMW']` filtert nur BMW-Fahrzeuge - Der gefilterte DataFrame kann wie gewohnt visualisiert werden - Zwei verschiedene Diagrammtypen zeigen unterschiedliche Aspekte

Verwendete Konzepte: - Datenfilterung mit Bedingung (Notebook 16b) - Kombination mehrerer Visualisierungen - Ableitung von Erkenntnissen

Lösung zu Aufgabe 15: Freie Analyse

Lösung anzeigen (Beispiel: Kraftstoffarten)

Aufgabe 15: Freie Analyse - Kraftstoffarten und Preis

Fragestellung: Wie unterscheiden sich die Kraftstoffarten im Preis?

Durchschnittspreis pro Kraftstoffart

```
preis_kraftstoff = df.groupby('fuelType')['preis_eur'].mean()
preis_kraftstoff = preis_kraftstoff.sort_values(ascending=False)
```

```
print("Durchschnittspreis nach Kraftstoffart:")
```

```
print(preis_kraftstoff.round(2))
```

Visualisierung

```
preis_kraftstoff.plot(
    kind='bar',
    title='Durchschnittspreis nach Kraftstoffart',
    xlabel='Kraftstoffart',
    ylabel='Durchschnittspreis (EUR)',
    color=['green', 'blue', 'orange', 'red'],
    figsize=(8, 5)
)
plt.tight_layout()
plt.show()
```

Erkenntnisse:

1. Hybrid-Fahrzeuge haben den höchsten Durchschnittspreis

2. Diesel und Benzinern liegen preislich ähnlich

3. Der Preisunterschied spiegelt vermutlich das Alter der Technologie wider

(Hybride sind tendenziell neuere Modelle)

Erklärung: - Eine eigene Fragestellung wird formuliert - Die passende Visualisierung (Balkendiagramm) wird gewählt - Die Ergebnisse werden interpretiert

Verwendete Konzepte: - Eigenständige Problemformulierung - Auswahl geeigneter Methoden - Kritische Interpretation

11 Troubleshooting

11.1 Häufige Fehler und Lösungen

Fehler	Ursache	Lösung
<code>FileNotFoundError</code>	CSV-Datei nicht gefunden	Prüfen Sie den Dateipfad; <code>used_cars.csv</code> muss im gleichen Ordner liegen
<code>KeyError: 'spalte'</code>	Spaltenname falsch geschrieben	Prüfen Sie die genaue Schreibweise mit <code>print(df.columns)</code>
<code>ModuleNotFoundError: pandas</code>	Pandas nicht installiert	<code>pip install pandas</code> im Terminal ausführen
<code>ModuleNotFoundError: matplotlib</code>	Matplotlib nicht installiert	<code>pip install matplotlib</code> im Terminal ausführen
Diagramm wird nicht angezeigt	<code>plt.show()</code> fehlt	Fügen Sie <code>plt.show()</code> nach dem Plot hinzu
Leeres Diagramm	Falsche Spaltennamen	Prüfen Sie <code>x=</code> und <code>y=</code> Parameter auf Tippfehler
<code>AttributeError: 'str' object</code>	Einzelne statt doppelte Klammern	Für mehrere Spalten: <code>df[['a', 'b']]</code> (doppelte Klammern)

11.2 Tipps zur Fehlerbehebung

1. **Spalten prüfen:** `print(df.columns)` zeigt alle verfügbaren Spalten
2. **Datentypen prüfen:** `print(df.dtypes)` zeigt die Typen aller Spalten
3. **Erste Zeilen anschauen:** `print(df.head())` gibt einen Überblick
4. **Schrittweise vorgehen:** Führen Sie Code Zeile für Zeile aus

12 Abschluss und Reflexion

12.1 Zusammenfassung

In dieser Übung haben Sie gelernt:

Konzept	Syntax	Anwendungsfall
CSV laden	<code>pd.read_csv('datei.csv')</code>	Externe Daten importieren
Spalte berechnen	<code>df['neu'] = df['alt'] * faktor</code>	Einheiten umrechnen, Werte transformieren
Häufigkeiten	<code>df['spalte'].value_counts()</code>	Kategorien zählen
Gruppierung	<code>df.groupby('kat')['wert'].mean()</code>	Durchschnitte pro Kategorie
Balkendiagramm	<code>.plot(kind='bar')</code>	Kategorienvergleich
Histogramm	<code>.plot(kind='hist', bins=n)</code>	Verteilungen analysieren
Scatter Plot	<code>.plot(kind='scatter', x=, y=)</code>	Zusammenhänge untersuchen
Liniendiagramm	<code>.plot(kind='line')</code>	Zeitliche Entwicklungen

12.2 Reflexionsfragen

Beantworten Sie für sich selbst:

1. Welche Visualisierung fanden Sie am aussagekräftigsten? Warum?
2. Was hat Sie an den Daten überrascht?
3. Welche weiteren Fragen könnten Sie mit diesem Dataset untersuchen?
4. Wo könnten Sie Datenvisualisierung in Ihrem Studium oder Beruf einsetzen?

12.3 Weiterführende Ideen

Falls Sie mehr üben möchten:

- Vergleichen Sie zwei Marken miteinander
- Untersuchen Sie den Zusammenhang zwischen Motorgröße und Verbrauch (mpg)
- Erstellen Sie eine Analyse für Fahrzeuge eines bestimmten Baujahrs
- Filtern Sie nach Getriebeart und vergleichen Sie die Preise

Herzlichen Glückwunsch! Sie haben diese Übungseinheit erfolgreich abgeschlossen. Sie können nun Daten mit Pandas laden, transformieren und mit verschiedenen Diagrammtypen visualisieren.